TECHNICAL OVERVIEW

Launch Crew

What the project does, the technology it is built with, and why each choice was made.

Repository: `launch-crew` (FastAPI backend + React client) · Document generated from the working tree, not from design notes · 11 October 2026

2	10	12	11	28
SERVICES	PIPELINE AGENTS	DESIGN STYLES	API ROUTERS	TEST MODULES

1. What the project is

Launch Crew is an AI product-creation pipeline that turns one free-text idea into a deployed marketing site. A user signs up, describes a business (or picks a template and refines it through a multi-step wizard), and a crew of specialised agents researches the idea, positions it, writes the copy, chooses a design direction, builds real HTML/CSS/JS, checks its own output *in a real browser*, and stops at a human approval gate. Only after the user clicks **Approve** does the Launcher deploy.

Everything after the build is work on a live artefact, not a screenshot:

- **Workspace** — the generated page is split into `index.html`, `styles.css`, `app.js` and a `README.md`, browsable, editable and downloadable as a zip.
- **Version control** — whole-file-set snapshots per run, named checkpoints, side-by-side diff, undo/redo and restore, with conflict detection when the page changed underneath you.
- **Plain-English editing** — a chat request is routed to the agent that owns that layer (copy, design or page) instead of regenerating everything.
- **Visual editor** — click an element in the sandboxed preview, locate it in source, get an AI proposal, then apply or reject it.
- **Audits** — on-demand security, SEO, accessibility, performance, dependency and generated-test passes, each with a repair path.
- **Live preview & sharing** — short-lived, revocable preview tickets and public share links.

2. How a run works

```
idea  ->  Researcher  ->  Strategist  ->  Copywriter  ->  Designer  ->  Engineer
                                                                          |
                                  +--- fixes <-- Critic <-- Playwright --+   (max 3 iterations)
                                  |
                        Panel  ->  awaiting_approval  ->  Launcher  ->  Netlify
                                                        (human click)
```

The orchestrator is a hand-written pipeline (`app/orchestrator/graph.py`, its own docstring opens "Custom orchestrator (no framework)"), not a third-party agent framework. It keeps an explicit `RunContext`, streams

every step to the client over SSE, enforces guardrails (`MAX_STEPS` , token budget, per-agent timeout, critic-iteration cap, tool-round cap) and persists run events so a refresh re-attaches to a run in progress.

- **Structured agents validate output** — Pydantic schemas with one corrective retry on bad JSON.
- **The Engineer returns raw HTML** — JSON-escaping a whole page is fragile, so it is sanitised and structurally checked instead.
- **Reviews are advisory, failures are not** — the Critic only calls the LLM when a browser check actually failed, keeping token cost proportional to real problems.
- **Deploys are atomic** — a single `awaiting_approval → deploying` transition prevents double or forged deployments.

3. Architecture and module map

LAYER	CONTENTS
Backend `app/`	`main.py` , `config.py` , `core/` (JWT, bcrypt, dependencies, rate limiting), `db/` (Mongo connection + indexes), `llm/` (Groq client, deterministic mock provider, mock edits), `agents/` (10 agents + schemas + the style registry + variants), `orchestrator/` (graph, state, runner, artifacts, readiness), `services/` (change impact, codebase index, preview, visual edit, workspace manager), `tools/` (browser checks, sanitizer, splitter, differ, deploy, web search, file writer, six audit scanners, secret scan, dependency scan, test generation), `api/` (11 routers), `prompts/*.md` .
Frontend `src/`	23 pages, 45 components, 4 hooks, a single `api.js` fetch wrapper, one 682-line token-driven stylesheet.
Data	MongoDB collections: `users` , `projects` , `runs` , `run_events` , `feedback` , `previews` — indexed on user/project/time, run/sequence (unique), share token, and a TTL index on preview expiry.
Artifacts	Generated HTML and screenshots go through one `Storage` abstraction: `mongo` (default, survives redeploys) or `local` disk.

Measured from the working tree: 72 Python modules under `backend/app/` (~11,100 lines), 5,257 lines of pytest across 28 test modules, 82 JavaScript modules in the client (~7,900 lines).

4. Backend technology, and why

TECHNOLOGY	ROLE HERE	WHY IT WAS CHOSEN
Python 3.10–3.12	Whole backend.	The agent, validation and HTML-analysis ecosystem (Pydantic, BeautifulSoup, Playwright) is strongest in Python. The pinned Playwright Docker image ships 3.12, so the code stays compatible across 3.10–3.12 rather than tracking the newest release.
FastAPI 0.115	REST API, SSE streaming, request validation, OpenAPI docs.	Native async matters when a single run holds many sequential LLM and browser calls; Pydantic-first validation means agent output and API payloads share one model layer; automatic `/docs` doubles as the API contract for the client.
Uvicorn	ASGI server (dev and containers).	The standard FastAPI runner; `uvicorn[standard]` adds the HTTP/1.1 and websockets extras without a separate web-server layer.
Pydantic 2.10 + pydantic-settings	Agent output schemas, request bodies, and all configuration from environment.	Structured LLM output needs a validator, not a parser. Model names deliberately have *no* defaults — they are configured in `.env`, because provider model IDs rotate faster than code changes. A blank value is treated as "unset" so an empty `MONGO_URI` in `.env` cannot wedge startup.
Motor 3.6 / PyMongo 4.9 + MongoDB 7	Persistence for users, projects, runs, run events, feedback, previews.	Run state is genuinely irregular — agent payloads, file-set snapshots, audit results and event logs differ per run — so a document store avoids a schema migration for every new field. Motor is the async driver, so no thread-pool bridging. The `run_events (run_id, seq)` unique index is what makes SSE replay and reconnect correct.
mongomock-motor	In-memory database when `MONGO_URI=mock://`.	The entire 28-module test suite runs without a Mongo service, which makes tests deterministic and CI cheap. The same index definitions still execute, so index regressions are caught.
Groq (via `groq` SDK)	LLM for all ten agents.	Chosen for low latency and a generous free tier, which matters for interactive runs. Three model slots (`GROQ_MODEL` , `GROQ_FAST_MODEL` , optional `GROQ_VISION_MODEL`) let cheap agents run on a cheaper model and the visual review be opt-in. A comma-separated key pool rotates round-robin and parks only the key that returned

		429, honouring `retry-after` where present, so one rate limit does not stall a run.
Offline `MockProvider`	`MOCK_LLM=true` produces a complete, deterministic run with no API key.	This is the single most consequential architectural choice: the pipeline, the browser checks, the approval gate, styles and tests are all verifiable offline, and CI never depends on a paid provider. Deploys are simulated and *labelled as simulated* rather than reported as real.
Playwright 1.49 + Chromium	Renders the generated page at 1280px and 375px and collects real metrics — contrast, font size, tap-target size, horizontal overflow — plus full-page screenshots.	"Tests its own work" is only true if a real layout engine is involved; static parsing cannot know whether text actually overlaps or whether a button is 44px. The design keeps the browser as a *metric collector* and makes every pass/fail decision in pure Python (`evaluate_metrics`), so the checks remain unit-testable without launching Chromium.
BeautifulSoup 4.12	DOM-level static analysis for the SEO, accessibility, security and dependency audits, and for the sanitizer.	Structural questions (missing alt text, absent meta, external script tags) are cheaper and more reliable against a parsed tree than against regex or a browser.
httpx	Async outbound calls: Netlify deploy API, Tavily search, preview probes.	Async client with timeouts and a response shape that keeps retry logic readable.
PyJWT 2.10 + bcrypt 4.2	Stateless bearer tokens (720-minute expiry) and password hashing.	No session table for a two-service deployment. bcrypt is the conventional, battle-tested choice; the code truncates to bcrypt's real 72-byte limit and the API schema caps passwords to match, so the boundary is explicit rather than a silent bug. OAuth was deliberately *not* implemented, because a half-wired provider would be a fake success path.
Custom rate limiter	In-process per-user run-creation limit (`RUNS_PER_HOUR`).	One Redis-free control is enough at this scale, and being in-process makes it directly testable. Documented as per-process — a multi-replica deployment would need a shared store.
Server-Sent Events	Live run trace to the browser.	The data flow is one-directional, so SSE is lighter than a websocket server and survives a refresh via the stored event log. Browsers cannot set headers on `EventSource` , so only the stream endpoint also accepts `?token=` ; everywhere else stays header-only.

Netlify API	Real deploys when `NETLIFY_AUTH_TOKEN` is set; an obviously flagged mock URL when it is not.	Static-hosting API needs no server on our side, matching generated static sites. A content-type probe diagnoses the classic "deployed page served as `text/plain`" failure instead of returning a bare error.
Tavily	Web search for the Researcher; falls back to mock search with no key.	A search API purpose-built for LLM agents keeps the Researcher's tool loop small (`TOOL_ROUNDS_CAP`) and budget-bounded.
pytest	All backend tests.	Fixtures and `monkeypatch` let the Chromium layer be replaced while the API, orchestrator, workspace, styles and version-control tests run against real code paths. Browser tests auto-skip when Chromium is absent, so the suite never fails for an environment reason it cannot fix.

5. The design registry: why output is not one template

A generator that produces the same centred-hero page in a different background colour is not a design system, so the styles are modelled as data and resolved per run.

MODULE	OWNS
`app/agents/styles.py`	Twelve `Style` records — each declaring its hero, navigation, feature layout, motif, corner radius, type tokens, spacing tokens, palette, font stacks, section set, section order and explicit rules. Plus 14 business types, section keyword matching, and the card payloads the UI reads.
`app/agents/variants.py`	Turns a style plus this run's brief into one machine-readable `DesignDirection`: brief-implied sections, the requested colour re-checked against WCAG AA, and final section order. Memoised, serialised onto run state as a plain dict, and rendered into a single `DESIGN_DIRECTION` block that both the Designer and the Engineer receive.
`app/llm/mock.py`, `mock_edits.py`	Render the direction as real markup offline, so "these two styles are genuinely different" is a testable assertion, not a manual inspection.

Why this matters operationally:

- **Revisions keep identity.** A stored direction is reused, so editing copy on a brutalist run does not silently re-skin it.
- **Contrast cannot be broken by a brief.** A requested colour that cannot reach AA on this style's surfaces is adjusted — and the reason is reported in the run trace as `color_note`, never silently dropped.
- **Frontends never duplicate truth.** Wizard, gallery, detail pages and the style picker all read `GET /api/styles`; `POST /api/styles/resolve` previews exactly what `POST /api/projects` will store — a test asserts the two agree.
- **Legacy runs still resolve.** Directions from before the style system (palette-only `theme_key` records) rebuild the page they originally produced.

6. Client technology, and why

TECHNOLOGY	ROLE HERE	WHY IT WAS CHOSEN
React 18.3	Whole UI.	The app is a set of stateful panels over one long-running job; React's model fits, and 18 is what the existing code and router version target. No framework switch was warranted.
Vite 5.4	Dev server and static build.	Zero-config ESM dev with instant HMR; `VITE_API_URL` is baked at build time, which is fine because it is a public value. Only three runtime dependencies keep the bundle honest.
react-router-dom 6.28	Public marketing routes, auth routes, and the authenticated app shell.	Nested layouts let the public site and the workspace share one tree without duplicating chrome.
Hand-written CSS custom properties	All 682 lines of design tokens, components and the dark theme via `:root[data-theme="dark"]`.	Token-driven CSS gives theme switching and consistent spacing without a utility-class build step or a component library's opinion. Critically: the *generated sites* are the product, so the app shell stays deliberately plain and avoids importing a framework's look.
Plain `fetch` wrapper (`api.js`)	Every backend call, bearer token injection, and a typed `ApiError`.	One small module is easier to audit than Axios plus interceptors plus a query library — and there are no server-cache semantics worth the dependency here.
Custom hooks	`useRunStream` (SSE with polling fallback), `useRun`, `useStartRun`, `useStyleRegistry`, plus toast/auth/theme.	The polling fallback is what makes run tracking survive a proxy that buffers SSE.
Custom SVG icon set + `LaunchCrewLogo`	All iconography.	A single inline icon component adds no network requests and no icon-pack version churn. One logo component — deliberately, so no alternative marks appear across the app.
No frontend test framework	—	Stated plainly rather than glossed over: client verification is `npm run build` plus a real browser pass against a throwaway backend. This is the weakest part of the toolchain and the highest-value thing to add.

7. Quality, safety and delivery

Guardrails around generated code

- HTML sanitizer strips external scripts, links, images and fonts, neutralises forms, blocks network calls and injects a CSP meta tag; previews render in a sandboxed iframe.
- API keys live in the backend only. The client bundle carries public configuration alone.
- Secret scanning and dependency scanning run as first-class audits rather than as a promise.
- Guardrail limits (steps, tokens, per-agent timeout, critic iterations, tool rounds, hourly runs) are settings, so a runaway run is bounded by configuration, not by luck.

Verification actually performed for the current state

- `pytest tests/test_wizard_api.py` → **23 passed** (registry integrity, twelve distinct slot tuples, resolve-equals-stored preview, parametrised 422s, back-compat briefs, per-run style override, compare quota behaviour, cross-user isolation).
- Thirteen targeted suites → **247 passed, 1 failed**. The failure is `test_orchestrator.py::test_launch_after_approval`: a real Netlify token in `backend/.env` makes the test hit the live API and take 429. Pre-existing and environmental, not a code defect; left unfixed and reported.
- `npm run build` → clean, 114 modules.
- Browser pass against a throwaway stack (`MOCK_LLM=true`, `MONGO_URI=mock://`, on ports 8001/5274): 12 style cards and 14 site types loaded, the review line printed the resolved direction, the launched page contained exactly the briefed sections and the requested AA-safe colour, compare produced two finished runs in different styles from one brief, and the console was clean. Tear-down removed the temporary build and both processes.

Deployment topology

ENVIRONMENT	HOW
Local, no keys	`docker compose up --build` — Mongo 7, backend image with Playwright Chromium (`shm_size: 1gb`, because Chromium needs more than Docker's default 64MB), and the client behind nginx.
Local, no Docker	`uvicorn app.main:app --reload` with `MONGO_URI=mock://` plus `vite dev`.
Production	`render.yaml` Blueprint for the backend (Starter plan or larger — Chromium will not fit in 512MB — with a persistent disk for artifacts), Vercel for the client, MongoDB Atlas for data.

8. Choices that were deliberately *not* made

INSTEAD OF	WHY THE SIMPLER THING WON
LangGraph / CrewAI / LlamaIndex	The pipeline is one linear path with one bounded loop and one human gate. A framework would add dependency weight and obscure the guardrails that make this auditable — and the file says so in its own docstring.
SQLAlchemy / an ORM	A document store plus targeted indexes. An ORM would fight the irregular run payload and add a migration step for every new agent output field.
Redis / Celery / a worker queue	Background tasks in-process with persisted run events. The queue's durability is reproduced by the event log, at zero extra infrastructure. Documented trade-off: a restart interrupts in-flight runs, and the code marks them failed rather than pretending they resumed.
A UI component library + Redux	Token CSS plus local hook state. A library's visual identity would compete with the generated sites it is meant to showcase.
Real OAuth, real payments, fake "success" buttons	Not implemented, and not faked. Unconfigured integrations report their status honestly (simulated deploys are labelled, missing keys surface as configuration states), because a fake success message is worse than a missing feature.
Building every page a brief mentions	The brief can carry a page list and the registry recommends pages per business type, but the builder produces a single page — see limitations.

9. Honest limitations of the current build

> These are the parts of the project that are genuinely not there yet. They are listed so the document describes the software, not the intention behind it.
>
> - **One page per run.** `spec.pages` is captured, recommended and previewed, but no route builds extra pages; the workspace is a single page split into four files.
> - **Regenerating one section in place** is partial: the visual editor locates and proposes a source patch, but there is no section-tree schema stored alongside the source, so structured add/duplicate/hide/reorder is not available.
> - **No animation system.** Motion exists only as hand-written CSS on the app shell; there is no preset library, live preview control surface, or per-section motion API. The generated pages do carry `prefers-reduced-motion` handling, which is the requirement every later addition must keep.
> - **No build executor and no per-project backend.** Generated output is static HTML/CSS/JS. There is no compile step, no dependency installation and no server-side runtime for a user's project, so a brief that needs a database or authentication gets scaffolding and instructions, not a working service.
> - **Plain-English feature coverage is narrow.** Chat edits route to copy, design or page agents; features needing real infrastructure (auth, checkout, bookings, uploads, webhooks, analytics) are not implemented by editing a static page.
> - **Rate limiting is per-process**, so it does not hold across replicas.
> - **Artifact durability depends on the storage choice**: `local` without a persistent disk loses previews on redeploy; S3/R2 is a documented swap point, not implemented.
> - **The client has no automated tests**, so regressions there are caught by the browser pass, not by CI.
> - **One known environmental test failure** (live Netlify token, §7) remains open.

10. Working with the code

```
cp backend/.env.example backend/.env && cp frontend/.env.example frontend/.env
docker compose up --build          # client :5173  ·  API :8000/docs

cd backend && pip install -r requirements.txt && playwright install --with-deps chromium
uvicorn app.main:app --reload      # MONGO_URI=mock:// needs no database server
pytest                             # browser tests auto-skip if Chromium is missing

cd frontend && npm install && npm run dev
```

Set `MOCK_LLM=true` to exercise the complete pipeline — real Playwright checks, real approval gate, real workspace and audits — with no API key and no cost. Set `GROQ_MODEL` / `GROQ_FAST_MODEL` from Groq's current model list; nothing is hardcoded in the codebase, by design.